Exp No: 26

TRANSACTION LEVEL MODELING (TLM) USING SYSTEMC

AIM:

To model and simulate a simple Transaction-Level Model (TLM) using Initiator and Target modules in SystemC.

PROCEDURE:

1. Create Initiator and Target modules using SystemC.
2. Declare initiator and target sockets for communication between the modules.
3. Implement the Initiator process to send integer data (555) using tlm_generic_payload.
4. Implement the Target's b_transport() function to receive the data and add a delay of 10 ns.
5. Bind the sockets, compile and execute the program, and observe the transaction output.

PROGRAM:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
using namespace std;

// Target
struct Target : sc_module {
tlm_utils::simple_target_socket<Target> socket;

SC_CTOR(Target) : socket("socket") {
socket.register_b_transport(this, &Target::b_transport);
}
```

```
void b_transport(tlm_generic_payload& trans, sc_time& delay) {
int* data = (int*) trans.get_data_ptr();
cout << "Target received: " << *data << endl;

delay += sc_time(10, SC_NS);
}
};

// Initiator
struct Initiator : sc_module {
tlm_utils::simple_initiator_socket<Initiator> socket;

SC_CTOR(Initiator) : socket("socket") {
SC_THREAD(thread_process);
}

void thread_process() {
tlm_generic_payload trans;
sc_time delay = SC_ZERO_TIME;

int data = 555;

trans.set_data_ptr(reinterpret_cast<unsigned char*>(&data));
trans.set_data_length(sizeof(data));

cout << "Initiator sending: " << data << endl;
socket->b_transport(trans, delay);
```

```
wait(delay);
}
};

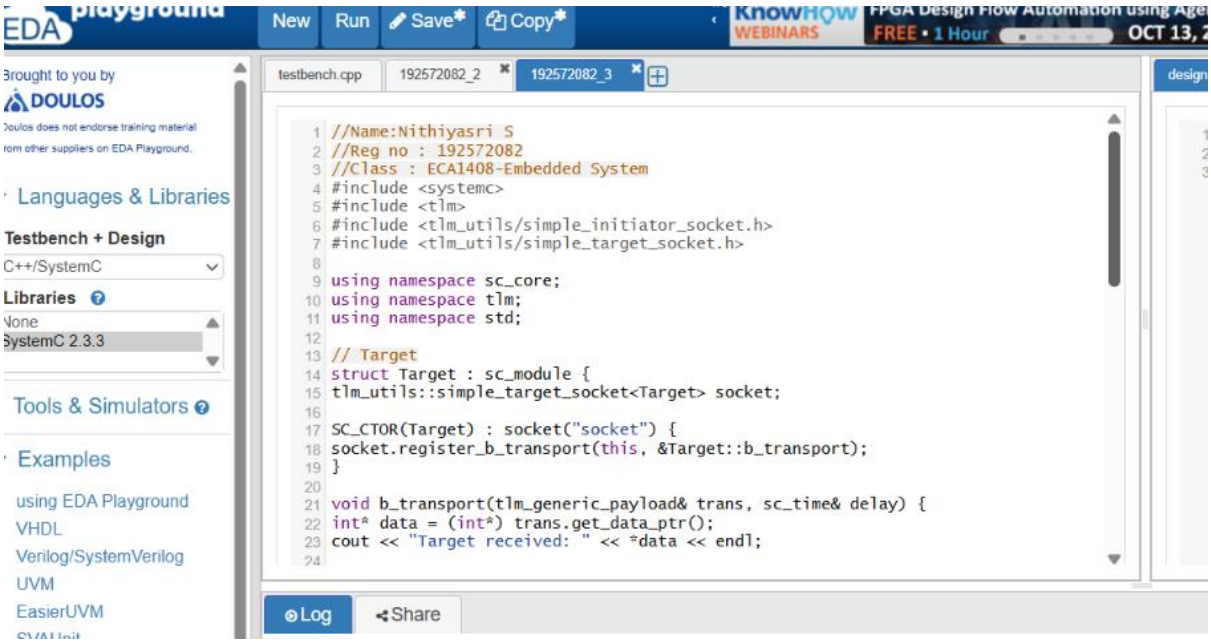
int sc_main(int argc, char* argv[]) {

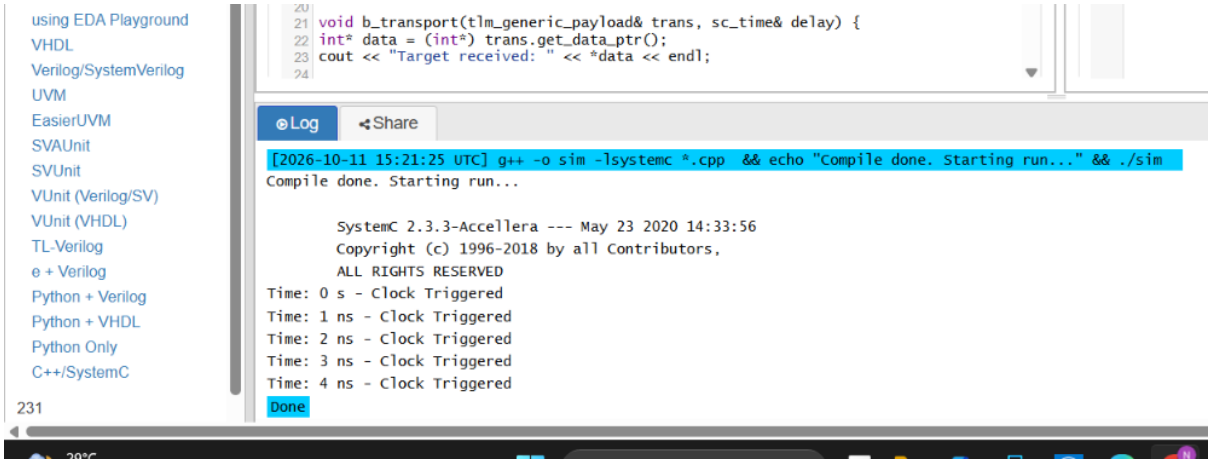
Initiator initiator("initiator");
Target target("target");

initiator.socket.bind(target.socket);

sc_start();

return 0;
}
```





RESULT:

Thus, the Transaction-Level Model using Initiator and Target modules was successfully implemented and simulated using SystemC.